

Progetto di reti Logiche

Filtro esponenziale fixed-point

Hajjioui Mohammed

-991018-

Introduzione

Il progetto prevede la realizzazione di un **filtro esponenziale del secondo ordine** per segnali digitali campionati. Il filtro calcola l'uscita $Y(t)$ a partire dall'ingresso $X(t)$ secondo la relazione:

$$Y(t) = \alpha \cdot X(t) + \alpha(1 - \alpha) \cdot Y(t - 1) + (1 - \alpha)^2 \cdot Y(t - 2)$$

con

$$\alpha = \frac{1}{2^k} \quad \text{con} \quad k \in [0, 7]$$

Il parametro k è fornito in ingresso al componente e si assume costante durante il funzionamento. Poiché la specifica non definisce il comportamento in caso di variazioni di k a run-time, il progetto adotta una scelta conservativa: **se viene rilevata una variazione di k** , gli stati interni del filtro vengono **riportati a zero**, così da ripartire da condizioni note ed evitare l'uso di stati non coerenti con i nuovi coefficienti.

Tutti i segnali numerici (ingresso, uscita e stati interni) sono rappresentati in formato **fixed-point Q16.16** su 32 bit complessivi (parte intera con segno + parte frazionaria).

Il filtro dispone di:

- un segnale di **reset** (RST), utilizzato all'avvio per portare il componente in uno stato noto e forzare a zero gli stati interni $Y(t-1)$ e $Y(t-2)$.

Ipotesi e vincoli

Nel seguito si assumono le seguenti condizioni operative:

- **Throughput**: il filtro elabora un campione per ciclo di clock.
- **Coefficiente k** : fissato all'avvio dell'elaborazione e non variato a run-time.
- **Overflow**: le operazioni aritmetiche sono eseguite su 32 bit in complemento a due; in assenza di saturazione esplicita, l'eventuale overflow produce un comportamento modulare.
- **Campionamento**: ad ogni fronte di salita del segnale di clock CLK viene acquisito un nuovo dato in ingresso.

Queste ipotesi definiscono il comportamento del componente e ne delimitano l'ambito di utilizzo.

Specifica

Il progetto è organizzato in una struttura gerarchica a moduli, con separazione tra:

- **calcolo dei termini** del filtro (a partire da ingressi e stati),
- **somma finale** dei termini per ottenere il nuovo campione filtrato,
- **memorizzazione degli ingressi e degli stati** per il passo temporale successivo.

Tutte le grandezze interne sono trattate a 32 bit in complemento a due (Q16.16) e le operazioni sui coefficienti 2^{-k} sono realizzate tramite **shift aritmetici**.

Riscrittura dell'equazione e ottimizzazione dei calcoli

Ponendo $\alpha=2^{-k}$, l'equazione del filtro può essere riscritta esplicitando i coefficienti:

$$Y_t = 2^{-k} X_t + (2^{-k} - 2^{-2k}) Y_{t-1} + (1 - 2 \cdot 2^{-k} + 2^{-2k}) Y_{t-2}$$

Dato che:

$$1 - 2 \cdot 2^{-k} + 2^{-2k} = 1 - 2^{-k} - (2^{-k} - 2^{-2k})$$

si ottiene una forma equivalente:

$$Y_t = 2^{-k} X_t + (2^{-k} - 2^{-2k}) Y_{t-1} + Y_{t-2} - 2^{-k} Y_{t-2} - (2^{-k} - 2^{-2k}) Y_{t-2}$$

Questa riscrittura consente di definire tre termini:

$$T_1 = X_t \gg k$$

$$T_2 = (Y_{t-1} \gg k) - (Y_{t-1} \gg 2k)$$

$$T_3 = Y_{t-2} - (Y_{t-2} \gg k) - [(Y_{t-2} \gg k) - (Y_{t-2} \gg 2k)]$$

e quindi:

$$Y_t = T_1 + T_2 + T_3$$

Riuso di termini (ottimizzazione)

Osserviamo che è possibile evitare di ricalcolare alcuni shift su $Y(t-2)$ salvandoli dal ciclo precedente:

- $Y(t-1) \gg k$,
- $T_2(t-1) = Y(t-1) \gg k - Y(t-1) \gg 2k$.

Di conseguenza, T_3 può essere calcolato come:

$$T_3 = Y_{t-2} - (Y_{t-2} \gg k) - [(Y_{t-2} \gg k) - (Y_{t-2} \gg 2k)] = Y_{t-2} - \underbrace{(Y_{t-2} \gg k)}_{\text{salvato}} - \underbrace{T_2(t-1)}_{\text{salvato}}$$

riducendo la logica necessaria nel percorso combinatorio (meno shift/add/sub ricalcolati sullo stato $Y(t-2)$).

Decomposizione in moduli

Term generator

Il modulo calcola i tre termini T_1 , T_2 , T_3 in **logica combinatoria** a partire esclusivamente da grandezze **già registrate** al precedente fronte di salita di **CLK**.

Gli ingressi utilizzati sono:

- X (campione registrato),
- k (parametro registrato),
- $Y1 = Y_{t-1}$ e $Y2 = Y_{t-2}$ (stati registrati),
- $Y1 \gg k$ salvato dal ciclo precedente,
- $T2_old$ salvato dal ciclo precedente.

In uscita produce:

- T_1 (dipendente da X e k),
- T_2 (dipendente da $Y1$ e k),
- T_3 (dipendente da $Y2$ e dai valori riutilizzati $Y1 \gg k$ e $T2_old$, come definito nella sezione precedente),
- inoltre produce il valore $Y1 \gg k$ calcolato nel ciclo corrente, destinato a essere registrato per il riutilizzo al ciclo successivo.

Term summer

Il modulo esegue la somma:

$$Y_{next} = T_1 + T_2 + T_3$$

State register

Raccoglie tutti i registri del sistema e gestisce:

- registrazione degli ingressi primari X e k ,
- registrazione dell'uscita Y ,
- aggiornamento degli stati del filtro $Y1$ e $Y2$,
- memorizzazione dei termini utili al riutilizzo:
 - $Y1 \gg k$,
 - $T2_old$,
- memorizzazione di $k(t-1)$ (valore precedente di k), utile al confronto per il rilevamento di variazione del coefficiente.

In caso di reset/inizializzazione (da **RST** o da variazione di k), lo state register riporta a zero gli stati $Y1$, $Y2$ e le grandezze memorizzate per il riutilizzo, così da ripartire da condizioni note.

Comparator

Il comparatore rileva la variazione del coefficiente confrontando i due valori registrati:

- $k(t)$,
- $k(t-1)$.

Il confronto è implementato tramite una rete di **XOR bit-a-bit** seguita da una **OR-reduction**: il segnale di uscita vale 1 se almeno un bit è diverso.

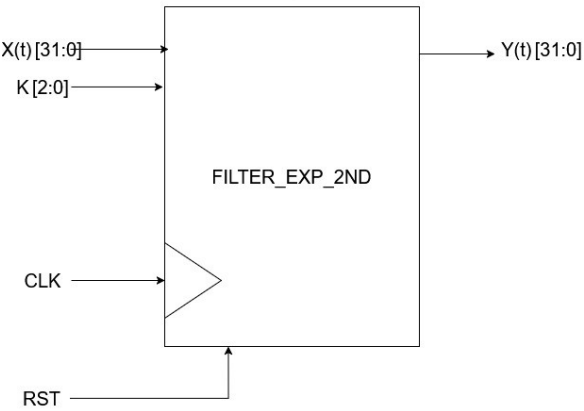
Quando la variazione viene rilevata, viene generato un segnale di controllo che provoca l'inizializzazione (azzeramento) degli stati nello **state register**.

Latenza attesa

Con questa architettura, ci si attende quindi una **latenza complessiva di 2 cicli di clock** tra l'applicazione di un nuovo campione in ingresso e la disponibilità del corrispondente risultato in uscita.

Nel caso in cui k venga modificato, il segnale di variazione viene rilevato confrontando valori già registrati e provoca l’azzeramento **dopo un solo ciclo di clock** a partire dalla variazione.

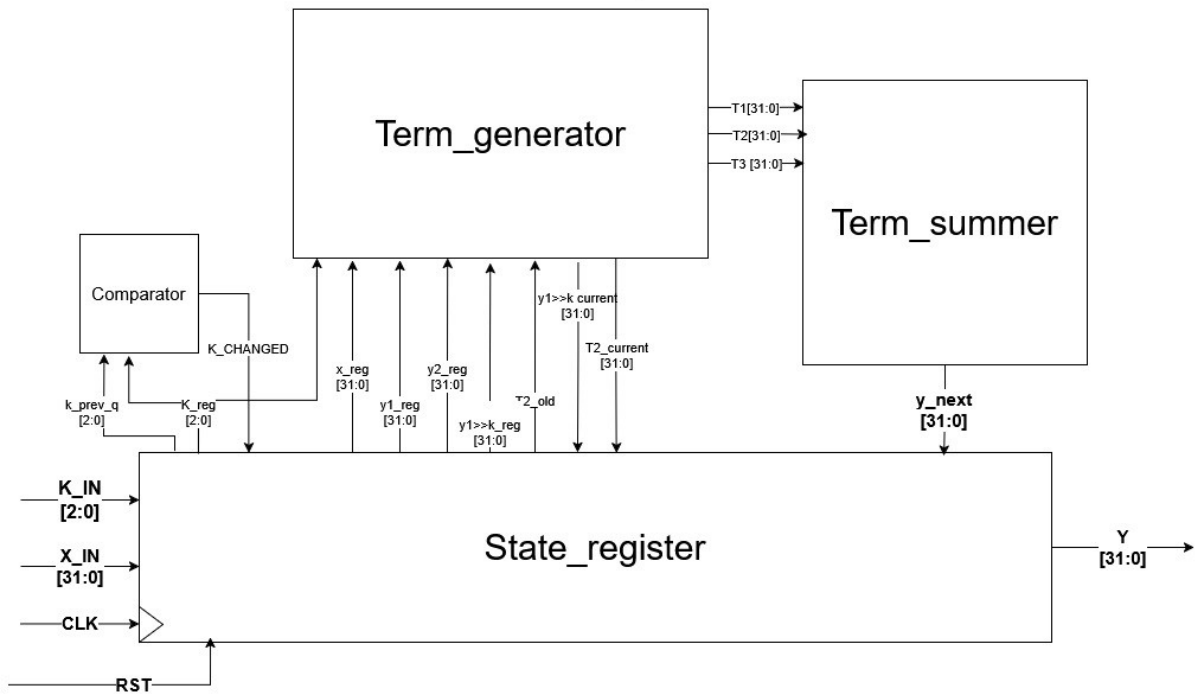
Interfaccia del sistema



Nome del segnale	Dimensione e codifica	Direzione
X	32 bit, Q16.16 complemento a 2	Input
Y	32 bit, Q16.16 complemento a 2	output
K	3 bit, intero senza segno	Input
RST	Singolo bit, attivo su 1	Input
CLK	Singolo bit	input

Ad ogni fronte di salita di CLK, se $RST = 0$, il filtro acquisisce il campione $X(t)$ e calcola il corrispondente campione filtrato $Y(t)$ con una latenza di due cicli di clk dovuta ai registri interni.

Architettura del sistema



L’architettura complessiva del filtro mostrata è organizzata in tre blocchi principali:

- **Term_generator**
- **Term_summer**
- **State_Register**

Questa scomposizione separa il calcolo dei termini del filtro, la somma finale e la gestione degli stati interni (inclusa la registrazione di ingressi/uscite), rendendo i moduli più semplici da comprendere e riutilizzare.

Il **Term_generator** è un blocco puramente combinatoriale che, a partire dai valori registrati (X , k , $Y1$, $Y2$) e dai valori salvati per il riuso ($Y1 \gg k$ e $T2old$), calcola i tre termini $T1$, $T2$, $T3$ secondo la riscrittura dell'equazione introdotta nella sezione precedente.

Inoltre produce $Y1 \gg k$ relativo al ciclo corrente, destinato a essere registrato nello **State_Register** per l'elaborazione del ciclo successivo.

Lo **State_Register** implementa tutti i registri del sistema:

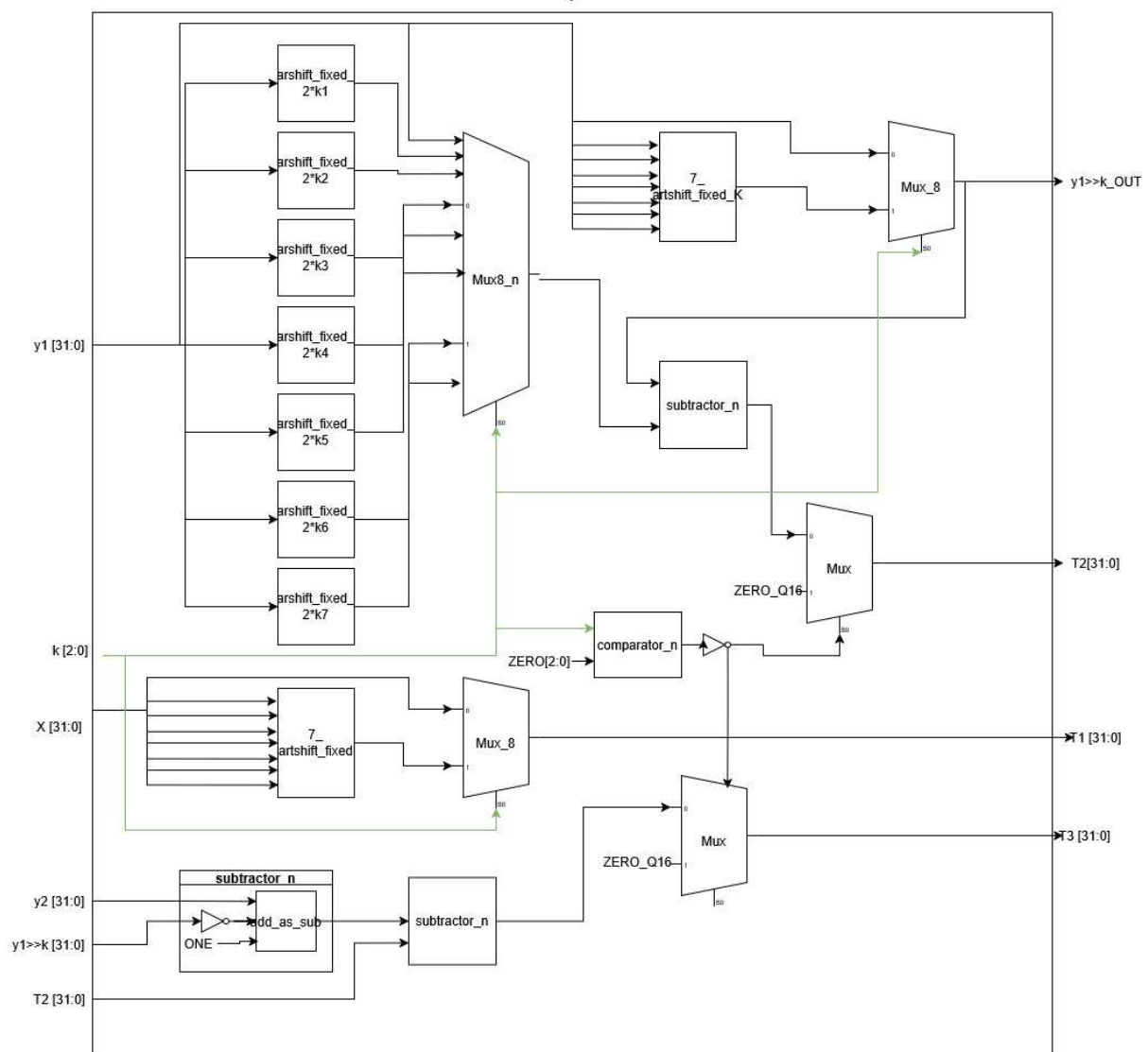
- registri di ingresso per X e k ,
- registro di uscita per Y ,
- registri di stato $Y1$ e $Y2$,
- registri per i valori riutilizzati $Y1 \gg k$ e $T2old$,
- registro per $k(t-1)$, usato per il confronto con $k(t)$ nel **comparator**.

Ad ogni fronte di salita di **CLK**, in condizioni operative normali, aggiorna l'uscita registrata e gli stati del filtro. In caso di reset (da **RST** o da segnale di variazione di k), azzera uscita e stati interni riportando il filtro in condizioni iniziali.

Term_summer

Il **Term_summer** realizza la somma dei tre termini prodotti dal **Term_generator**; la somma è implementata strutturalmente tramite addizionatori in cascata. L'uscita Y_next è combinatoriale e viene registrata dallo **State_Register**.

Modulo: Term_generator



Il modulo **Term_generator** opera esclusivamente su grandezze già registrate al precedente fronte di salita di **CLK** e utilizza come ingressi il parametro **k**, il campione **X**, gli stati **Y1** e **Y2**, oltre ai valori memorizzati per il riuso dal ciclo precedente **Y1>>k** e **T2_old**. In uscita fornisce **T1**, **T2**, **T3** e il valore **Y1>>k** calcolato nel ciclo corrente, destinato a essere registrato per l'elaborazione del ciclo successivo.

Il termine **T1** viene ottenuto applicando uno **shift aritmetico a destra** di ampiezza variabile al campione **X** in funzione di **k**. Analogamente, per il calcolo di **T2** è necessario produrre due versioni shiftate di **Y1**, ovvero **(Y1>>k)** e **(Y1>>2k)**, e combinarle mediante sottrazione. Il termine **T3** è calcolato combinando lo stato **Y2** con i contributi derivati da **Y2>>k** e **T2_old**.

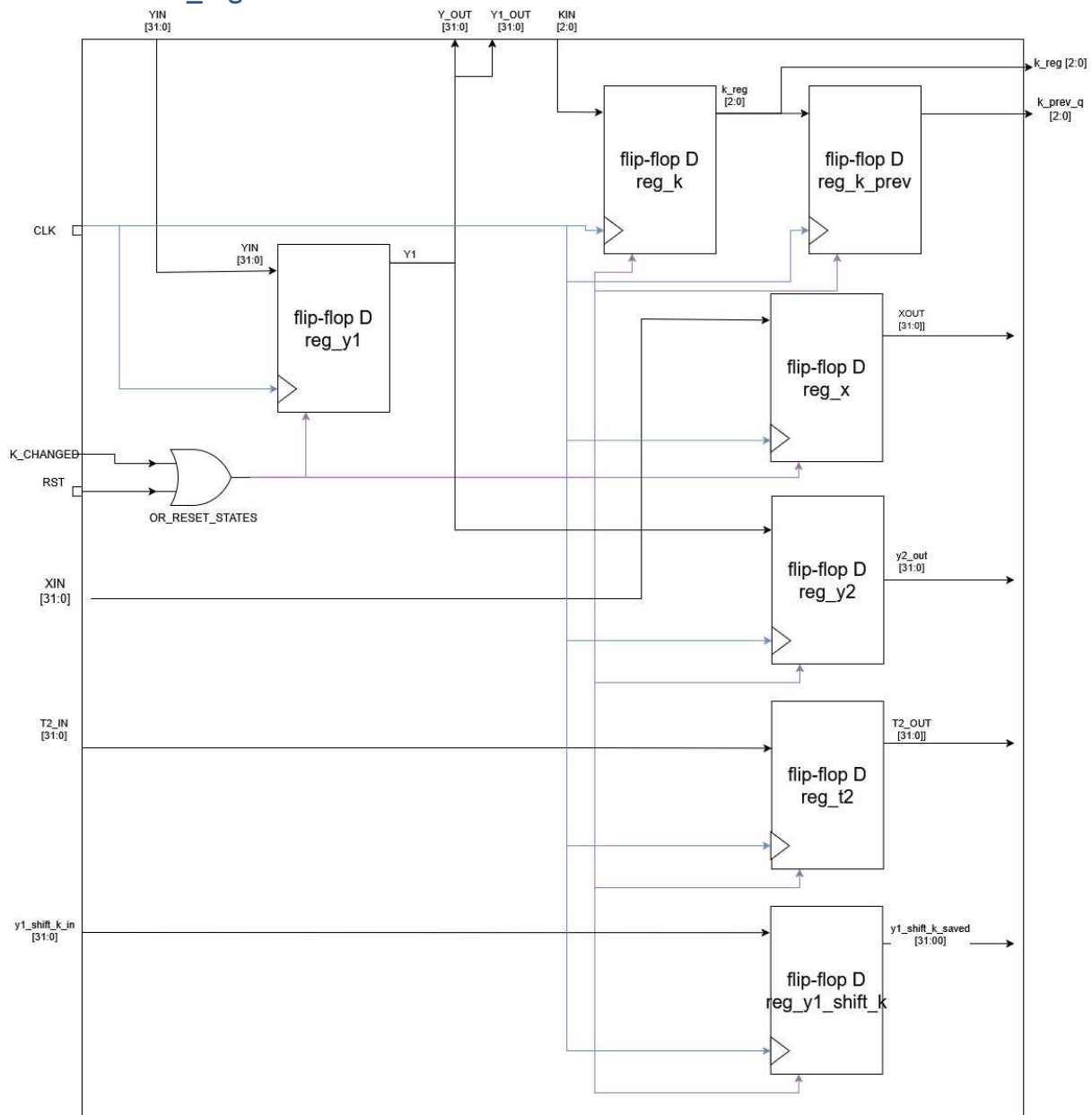
Dal punto di vista architetturale, il modulo è realizzato come rete combinatoriale composta da sotto-moduli parametrici (tramite generic **N**), riutilizzabili anche in altri contesti:

- **Shifter a ampiezza costante (arshift_fixed_n)**: implementa uno shift aritmetico a destra di ampiezza fissata in fase di generazione. È utilizzato per precomputare, in parallelo, tutte le possibili versioni shiftate richieste.

- **Selezione dello shift variabile tramite multiplexer (mux8_n):** per realizzare lo shift variabile (0 ... 7) richiesto da k, il Term_generator calcola in parallelo gli otto candidati (shift fissi) e seleziona il corretto valore con un **mux a 8 vie**. Il mux8_n è realizzato gerarchicamente a partire dal componente elementare mux2_n su tre livelli di selezione, favorendo il riuso e mantenendo il progetto strutturale.
- **Sottrazioni (subtractor_n):** le differenze necessarie al calcolo di T2 e di T3 sono ottenute tramite sottrattori dedicati. Il subtractor_n è implementato mediante complemento a due ($\sim B + 1$) e riusa l'addizionatore parametrico adder_n.
- **Addizionatore parametrico (adder_n):** realizzato come ripple-carry a (N) bit, composto da una catena di full_adder_1b istanziata con costrutto generate, e impiegato internamente nei sottrattori.
- **Gestione del caso (k=0):** quindi $\alpha=1$ il filtro deve degenerare a $Y(t)=X(t)$. In tale condizione i contributi associati agli stati devono annullarsi; il Term_generator forza quindi i termini T2 e T3 a zero tramite un mux2.

Questa organizzazione consente di implementare i termini del filtro riducendo il lavoro combinatorio necessario grazie al riutilizzo dei valori già calcolati nel ciclo precedente.

Modulo: State_register



Il modulo **State_register** implementa l'insieme dei registri del sistema e svolge due funzioni principali: **registrazione degli ingressi/uscite** del componente top-level e **memorizzazione degli stati interni** necessari alla relazione ricorsiva del filtro, inclusi i valori salvati per il riuso nel ciclo successivo.

Il modulo contiene i seguenti registri (tutti sincroni al fronte di salita di **CLK**):

- **Registro di ingresso X:** memorizza il campione $X(t)$ (formato Q16.16, 32 bit).
- **Registro di ingresso k:** memorizza il parametro $k(t)$ (3 bit).
- **Registro $k(t-1)$:** memorizza il valore precedente di k , utile al confronto per rilevare eventuali variazioni del coefficiente.
- **Registri di stato del filtro:**
 - $Y1 = Y(t-1)$ (**nota:** questo registro è utilizzato anche come **registro di uscita Y**),

- $Y2 = Y(t-2)$,
- **Registri per il riuso dei termini** (ottimizzazione):
 - $(Y1 \gg k)$: valore calcolato nel ciclo corrente e riutilizzato al ciclo successivo come $Y2 \gg k$,
 - $(T2_old)$: termine $T2$ calcolato nel ciclo precedente, riutilizzato nel calcolo di $T3$.
- **Registro di uscita Y** : coincide con il registro $Y1$ (cioè $Y \equiv Y1$), fornendo l'uscita del filtro in forma registrata.

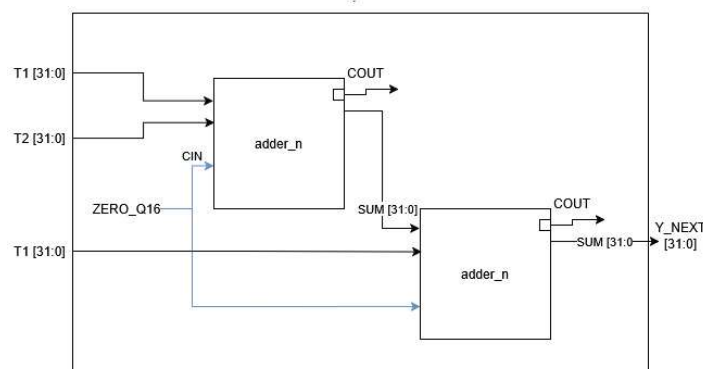
L'utilizzo dello stesso registro per Y e $Y1$ costituisce un'ottimizzazione architetturale: riduce il numero di registri complessivi e mantiene compatta la catena di aggiornamento degli stati, evitando l'introduzione di un ulteriore ritardo sull'uscita e sugli stati interni.

Il modulo gestisce inoltre l'inizializzazione degli stati. In presenza di **RST** oppure quando viene rilevata una **variazione di k** (segnale $k_changed$ generato dal comparatore), viene attivato un reset condizionale che **azzerà gli stati interni** $Y1$, $Y2$ e i registri associati al riuso, riportando il filtro in una condizione nota. Questa scelta garantisce coerenza dell'elaborazione nel caso in cui il coefficiente venga modificato.

In condizioni operative normali (assenza di reset), ad ogni fronte di salita di **CLK**:

- gli ingressi vengono acquisiti nei rispettivi registri,
- l'uscita (Y) viene aggiornata con il nuovo valore Y_next calcolato dalla logica combinatoriale,
- gli stati vengono avanzati secondo la catena $Y2 \leftarrow Y1$, $Y1 \leftarrow Y$,
- i valori $Y1 \gg k$ e $T2_old$ vengono registrati per l'iterazione successiva.

Modulo: Term_summer



Modulo: Term_summer

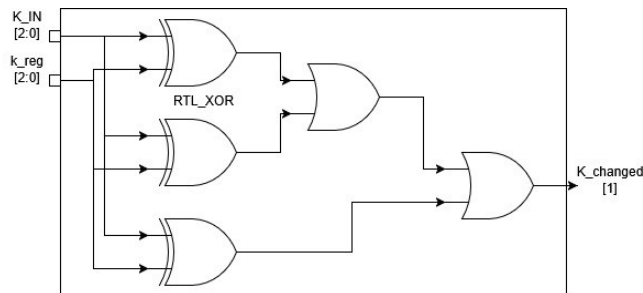
Il modulo **Term_summer** esegue la somma finale dei contributi calcolati dal **Term_generator**, producendo il valore combinatoriale Y_next .

L'architettura è interamente strutturale e realizza la somma tramite **due addizionatori a (N) bit in cascata**:

1. calcolo intermedio $S_{12} = T1 + T2$)
2. calcolo finale $Y_{next} = S_{12} + T3$)

Gli addizionatori impiegati sono componenti parametrici (adder_n) che implementano una somma bit-a-bit tramite catena di full adder; i segnali di carry di uscita non sono utilizzati per la generazione dell'uscita (comportamento modulare su (N) bit). Il valore Y_{next} viene successivamente **registrato** nello **State_register**.

Modulo: comparator



Il modulo **Comparator** ha il compito di rilevare condizioni di uguaglianza/diseguaglianza tra segnali a (N) bit. Nel sistema viene utilizzato in due punti:

- **Rilevamento variazione di k:** confronta $k(t)$ e $k(t-1)$ (entrambi registrati) e genera il segnale $k_changed$, pari a 1 quando almeno un bit differisce.
- **Gestione del caso $k=0$ nel Term_generator:** confronta k con il valore costante "000" per individuare la condizione ($k=0$), usata per forzare i termini che devono annullarsi.

L'implementazione è strutturale: si esegue lo XOR bit-a-bit tra i due ingressi e si effettua una OR-reduction dei risultati; l'uscita vale 1 se almeno uno XOR è pari a 1.

Verifica

La verifica del sistema viene condotta mediante **simulazione** del componente top-level. L'approccio di testing prevede:

- **Stimolazione sincrona** degli ingressi, con valori applicati in modo da rispettare la stabilità attorno al fronte attivo di clock (setup/hold), così da riprodurre condizioni realistiche di funzionamento.
- **Osservazione delle waveform** per validare il comportamento dinamico del filtro (in particolare la risposta a variazioni a gradino e sequenze di ingresso), verificando qualitativamente l'andamento atteso al variare del parametro k .
- **Assertion semplici** per controllare proprietà fondamentali e non ambigue, come il corretto effetto del reset e il ripristino di condizioni note quando richiesto (reset esterno e re-inizializzazione dovuta al cambio di k).
- Verifica finale sul modello **Post Place & Route**, così da includere i **ritardi** e validare il corretto funzionamento anche in presenza di timing realistico.

Test-bench

Il test-bench sviluppato per la verifica del sistema **top-level** ha il compito di generare automaticamente il segnale di clock e gli stimoli di ingresso X, K e **RST**, monitorando l'uscita Y per validare il comportamento complessivo del filtro.

Il test-bench è strutturato nei seguenti blocchi logici:

- **Clock generator**: produce un clock periodico con periodo **20 ns** (frequenza **50 MHz**).
- **Stimulus process**: applica le sequenze di stimolo agli ingressi X e K e gestisce l'asserzione di **RST** secondo gli scenari di prova.
- **Monitor/Check**: riporta informazioni tramite messaggi report e include assertion semplici per verificare proprietà fondamentali (ad esempio (Y=0) durante reset e ripartenza da condizioni note dopo re-inizializzazione).

Temporizzazione degli stimoli

Per evitare condizioni non realistiche in simulazione e garantire margini rispetto al fronte di campionamento, gli ingressi vengono aggiornati **lontano dal fronte attivo di clock**, tipicamente a **un quarto di periodo** dal fronte di salita. In questo modo X e K risultano stabili prima del fronte di salita successivo.

Le verifiche tramite assertion vengono eseguite **dopo il fronte di salita**, quando l'uscita registrata (Y) ha avuto il tempo di stabilizzarsi (clock-to-out).

Modalità di verifica della correttezza

La correttezza del sistema viene verificata con una combinazione di:

- **osservazione delle waveform**, per valutare l'andamento dinamico del filtro ;
- **assertion mirate** su condizioni non ambigue, in particolare:
 - (Y=0) durante **RST**,
 - ripartenza da condizioni note quando viene applicata una re-inizializzazione (reset esterno o dovuto a variazione di k).

Gli stimoli sono espressi in formato Q16.16, utilizzando costanti rappresentative come:(1.0), (0.5), (-1.0).

Simulazioni eseguite

La verifica viene condotta in simulazione behavioral; viene ripetuta anche in simulazione Post Place & Route (timing simulation) per includere i ritardi di propagazione e verificare che l'uscita (Y) si aggiorni coerentemente con le temporizzazioni fisiche del circuito.

Casi d'uso

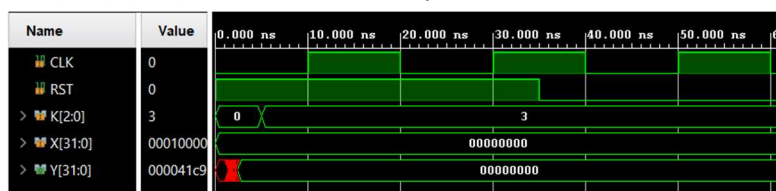
Caso d'uso 1 — Reset

Obiettivo: verificare l'inizializzazione del sistema.

Stimoli: (RST=1) per alcuni cicli; (X=0); K fissato.

Atteso: durante (RST), (Y=0); al rilascio il filtro riparte da stati azzerati.

Waveform:



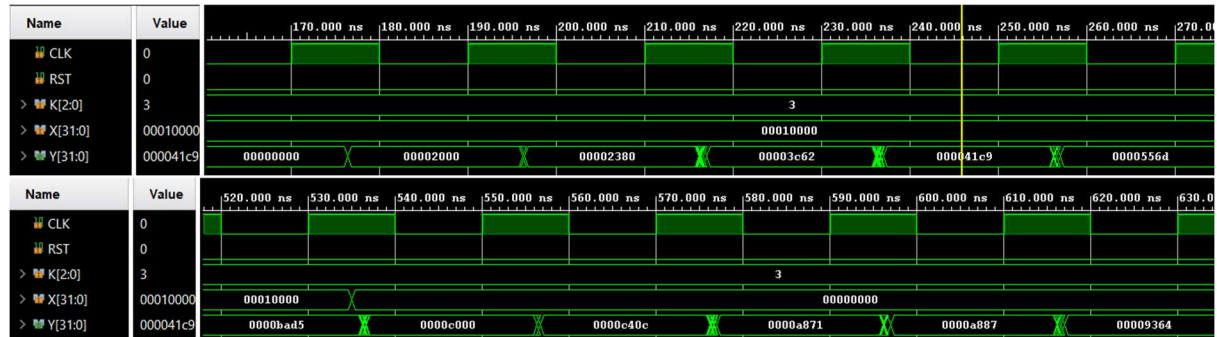
Caso d'uso 2 — Risposta al gradino (k=3)

Obiettivo: osservare la risposta del filtro a un gradino con smoothing intermedio.

Stimoli: (K=3); (X: 0 $\rightarrow$ +1.0 $\rightarrow$ 0).

Atteso: (Y) converge verso il valore finale e poi rientra verso 0 con andamento coerente con (k).

Waveform:



Caso d'uso 3 — Caso limite k=0 (pass-through)

Obiettivo: verificare il comportamento passa-tutto e la latenza dell'architettura.

Stimoli: (K=0); (X: 0 $\rightarrow$ +1.0 $\rightarrow$ 0).

Atteso: (Y) segue (X) (latenza osservabile in waveform).



Caso d'uso 4 — Cambio K durante funzionamento

Obiettivo: verificare la gestione della variazione di (k) (re-inizializzazione stati).

Stimoli: (X) costante e (k=1) $\rightarrow$ (k=2).

Atteso: dopo il cambio di (k) gli stati vengono azzerati al primo fronte utile e il filtro riparte da condizioni note.

